DevOps, Software Evolution and Software Maintenance (MSc)
KSDSESM1KU
IT-University of Copenhagen

Marek Joukl(jouk@itu.dk), Rafael Rohal (rafro@itu.dk)
Benjamin Storm Larsen (bsla@itu.dk), Malte Sauerland-Paulsen (saue@itu.dk)

May 2026

Contents

1 Introduction

ITU-MiniTwit is a Twitter-like microblogging service from the original Python Flask codebase that this course provides. Over the semester, we refactored it into a Go application with a PostgreSQL database and deployed in containers on a single-node Docker Swarm hosted at Hetzner. The application infrastructure can be seen in figure1. We had three main motivations for picking Go: We wanted to challenge ourselves by learning a new language, prioritized the performance of a compiled language, and the static typing of Go provided a good contrast to Python's dynamically typed syntax. The rest of this report describes how we provisioned, built, deployed, observed, and secured the system.

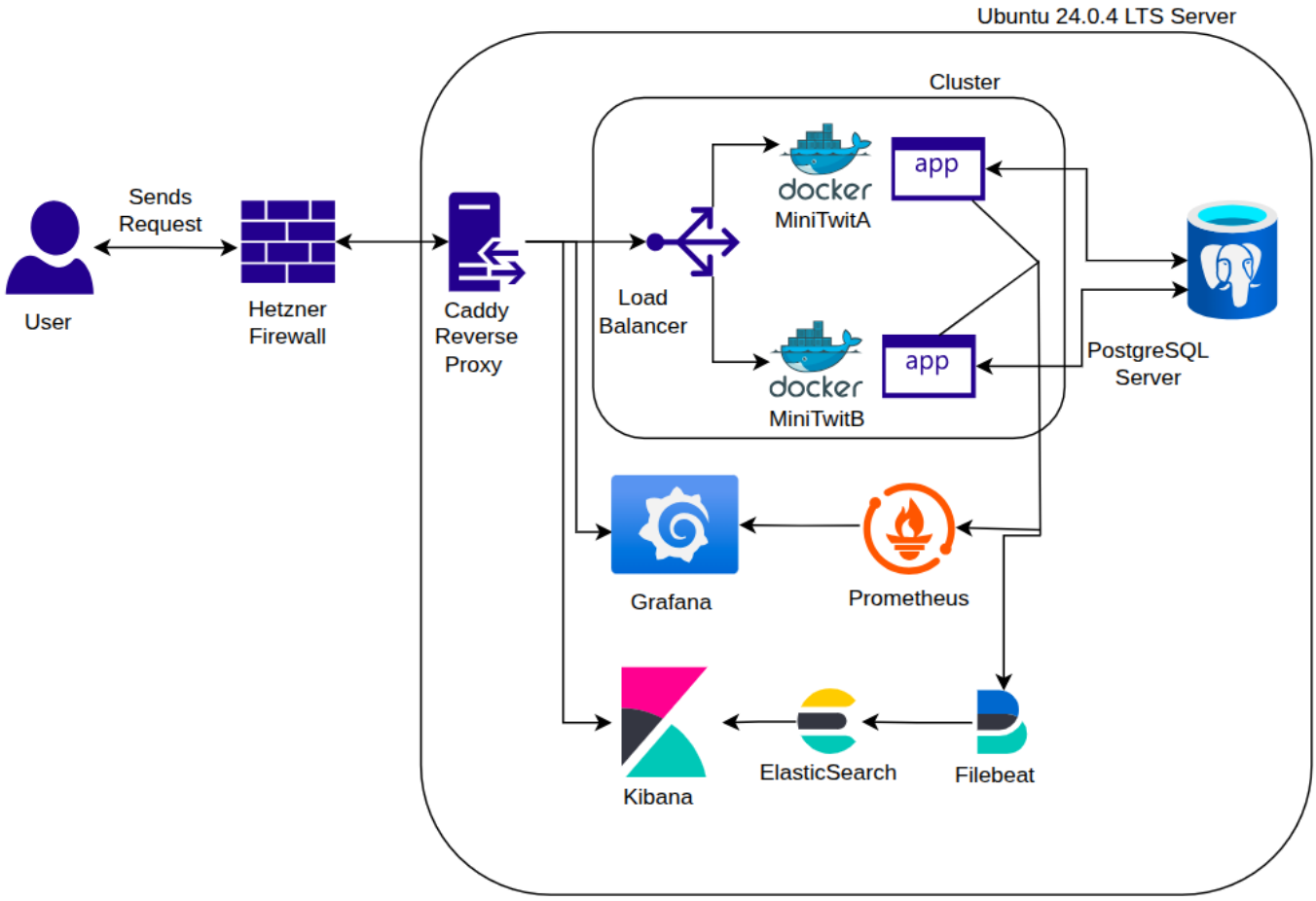

Figure 1: Minitwit System Overview

2 Provisioning

2.1 Hosting provider

We chose to rent a VPS from Hetzner due to its strong balance between affordability and performance. Hetzner is well known for offering cost-effective virtual private servers with reliable hardware specifications, making it a suitable choice for hosting our application within a limited budget.

Hetzner's built-in firewall was another key factor. It lets us apply firewall rules at the infrastructure level, filtering unwanted traffic before it reaches the server. After completing the migration and application setup, we updated our Go provisioning script to automatically create and attach the firewall to each server.

2.2 VPS

Converting the legacy system from Python to Go significantly improved performance, with faster request handling, lower CPU usage, and reduced memory consumption. This allowed the application to run efficiently on less powerful hardware

while handling a similar workload.

We chose a Hetzner CX23 server with 2 vCPUs, 4 GB RAM, and 40 GB storage. Multiple vCPUs were necessary to support concurrent applications, while 4 GB RAM ensured stable performance without heavy swap usage. The server also includes 20 TB monthly egress, offering strong cost efficiency at just €3,99 per month.

2.3 Provisioning the server

We chose to define our infrastructure provisioning logic as code rather than relying on Hetzner's GUI. Hetzner provides an official Go library that acts as a wrapper around their API, which we used to automate the provisioning of the server directly from our application code.

Terraform was also considered as a potential solution for infrastructure provisioning. However, we ultimately preferred the simplicity and maintainability of using a single Go script. This decision aligned naturally with the broader migration of the main application from Python to Go, allowing the team to remain within a familiar ecosystem and programming language.

2.4 Bootstrapping

Server bootstrapping is a critical part of deployment, as misconfigurations can cause downtime, unstable services, and security vulnerabilities. To reduce these risks, we used Ansible for automated server provisioning and configuration.

Ansible is widely adopted for infrastructure automation and offers an idempotent design, allowing playbooks to run repeatedly without creating inconsistencies. This ensures the server setup remains reproducible, maintainable, and consistently updated while reducing manual errors and improving security.

The setup consists of two Ansible playbooks. The first handles server bootstrapping and security hardening, including system configuration, security measures, and deployment preparation.

Table 1: Security hardening measures applied during server bootstrapping

Security Measure	**Description**
Non-root administrative user	Creates a user without direct root access while granting `sudo` privileges.
System upgrades	Updates and upgrades server packages and system components.
Disable root login	Prevents direct root login to improve security.
System hardening	Applies Linux kernel, SSH, and network hardening settings.
UFW and Fail2Ban	Configures firewall rules and intrusion prevention mechanisms.
Auditd setup	Enables system auditing and security event logging.
Unattended upgrades	Configures automatic installation of security updates.

The second playbook focuses specifically on installing and configuring Docker along with the Docker Compose plugin. Separating these responsibilities into distinct playbooks improves maintainability and makes the provisioning process easier to understand and manage.

3 CI/CD

3.1 Choice of a CI/CD system

Since our project is hosted in a GitHub repository, the most natural choice for our CI/CD system was GitHub Actions. It is also the de facto standard in the industry, with the skills learned being transferable to other CI/CD systems.

3.2 Pipeline

Workflows are located in `.github/workflows/`. The main pipeline, `ci.yml`, runs on every push and pull request targeting `main`. The first four jobs run in parallel, the fifth (`uitest`) job depends on them all completing. All five must pass before the `deploy` job runs:

1. `gofmt`: uses `gofumpt` to check whether every Go file in the repo follows its formatting rules
2. `golangci`: runs the `golangci-lint` linter on the Go code to check for bugs, style problems, and common mistakes
3. `hadolint`: `Dockerfile` linter

4. `integration_test`: runs Go integration tests that make HTTP requests to verify the endpoints work correctly
5. `uitest`: runs Selenium UI tests to verify that the user interface works as intended
6. `deploy`: deploys to production, but only after all previous jobs pass
7. `codeql`: runs GitHub's CodeQL static analyzer on the Go code to check for security vulnerabilities
8. `image_scan`: builds the production container image and runs Trivy on it to check for fixable HIGH/CRITICAL vulnerabilities in OS packages and Go dependencies

The `deploy` job additionally requires `github.ref == refs/heads/main` – meaning that pull requests run the full pipeline but only merges into main actually deploy.

3.3 Deployment

The deploy job SSH-es into the server as a dedicated `deploy` user using a private key stored in GitHub Secrets. The `deploy` user is a member of the `docker` group, which allows the user to run docker containers without `sudo` privileges. The `deploy` user then loads environment variables from `.env`, pulls the latest `main`, builds the Docker image, deploys the stack using `docker-compose.yml`, and updates the web service with the new image.

3.4 Releases

We use a `weekly-release.yml` workflow that runs every Friday at 9:00 UTC, and creates a GitHub Release tagged `release-YYYY-MM-DD` with auto-generated release notes with every merged pull request since the previous release tag.

4 Monitoring

4.1 Design and architecture

Our metrics pipeline has three components: the Go web service produces metrics (`GET /metrics`), Prometheus pulls and stores them, and Grafana visualizes them. A fourth service, node-exporter, exposes host-level metrics (CPU, memory, disk). We used the official `prometheus/client_golang` library. Our HTTP middleware automatically records request count, latency, and status code for every route. The counters for specific stats are incremented separately inside the relevant handlers and exposed as, for instance, `new_registered_users`. Prometheus then scrapes the metrics every 15s, and Grafana pulls and visualizes them.

The Grafana dashboard is fully provisioned from a folder in our repository, which defines what the dashboard looks like. It is set up automatically after deployment, with the tradeoff that any changes from the UI will not persist. Lastly, Caddy exposes Grafana at `grafana.kulturbase.dk` over HTTPS.

4.2 What we monitor

The main metrics we monitor are request rate, error rate, latency, and success rate. The reason behind this is that it immediately tells us whether something is wrong and, if so, points us in the right direction. The second part of the dashboard focuses on overall health and serving capabilities. The main visualizations show CPU load, health checks, and an overview of activity on the server. This enables us to distinguish application issues from infrastructure issues and determine whether the app is being actively used.

4.3 Reflections

The monitoring setup needs to take the deployment strategy into consideration. When switching from `docker compose` to `Docker Swarm` with multiple replicas, our scraping silently broke. Each scrape hit a different replica through Swarm's load-balancing virtual IP, so counters appeared to reset between scrapes. The solution was to use Prometheus DNS service discovery to scrape each replica separately and then sum the results in Grafana.

5 Logging

5.1 Design and architecture

Our logging pipeline uses an EFK stack (Elasticsearch, Filebeat, Kibana), as presented in the exercises. The Go web service writes structured JSON logs through `slog` to stdout. Docker captures container stdout into `/var/lib/docker/containers/`

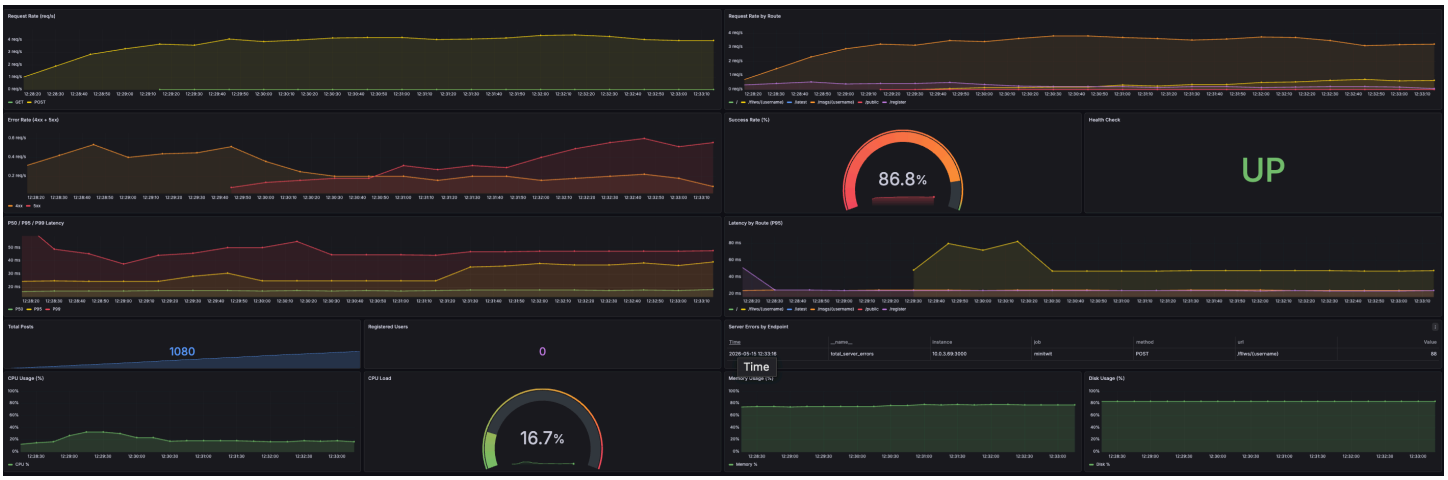

Figure 2: Grafana dashboard overview

via its default JSON-file driver. Filebeat runs one instance per node in Swarm, tails those log files, decodes the Docker JSON, and ships entries directly to Elasticsearch, which stores and indexes them. Kibana sits in front for search and visualization.

The whole stack runs on its own ELK overlay network, with Caddy exposing `kibana.kulturbase.dk` over HTTPS.

5.2 What we log

The Go application produces structured JSON logs at multiple levels through `slog`. The HTTP middleware automatically logs every request with method, route, status, duration, and client IP. Handler-level `Logger.Error` calls add explicit context when something fails (database query, template render, JSON decode).

Container stdout from supporting services (Postgres, Caddy, Prometheus) is also captured automatically. In Kibana, we built a dashboard with four saved searches: a live error stream, a failed-request stream (`status >= 400`), a user-activity stream (registrations, logins, posts), and overall log volume by service.

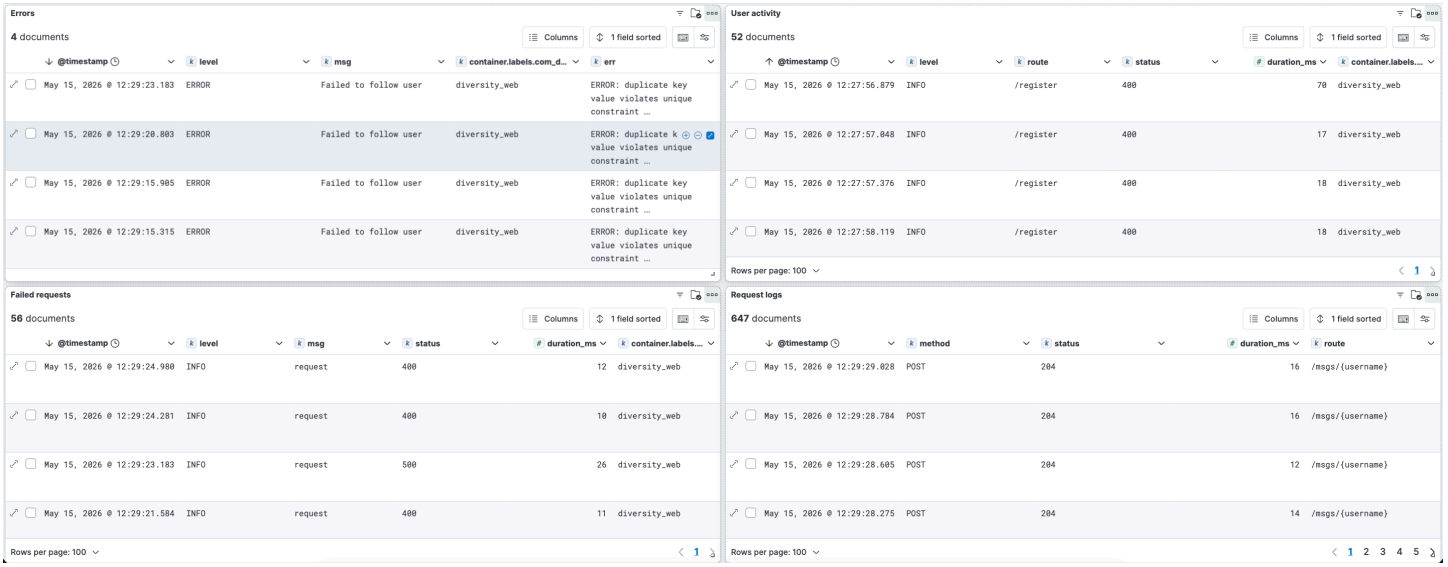

Figure 3: Kibana dashboard overview

5.3 Reflections

Choosing the EFK stack over Grafana + Loki was, in our opinion, the wrong call for a project this small. Elasticsearch alone consumes a large amount of memory, and the stack was tricky to set up. For example, Filebeat refused to start until its configuration file was owned by root with the correct permissions, which broke our deployments until we fixed the file ownership manually on the server.

Loki would have allowed us to keep everything under one Grafana frontend, removing Kibana as a separate system to configure and manage access to. Even though it introduced unnecessary complexity, the decision was still worth it as a learning experience, since we got to try the industry-standard logging stack and encounter the kinds of real-world problems that come with it.

For a project of this size, it was overkill, but as a one-time exercise to engage with heavier tooling, it was a good choice. It also nicely separated the two concerns: Grafana tells us whether something is wrong, while Kibana explains what happened.

6 Swarm

As mentioned earlier in the report, we were limited by cost, which is why we opted to setup a non-HA Swarm cluster for our server.

The old CI/CD setup would tear down the entire compose application, including the database, monitoring services, and the application itself. This caused brief downtime for users and also left the application vulnerable to potential issues in newly updated Docker images. If a new image contained an error that prevented it from starting, the setup would be unable to revert to the previously working version of the application, resulting in extended downtime. This was far from ideal.

Instead, we migrated the application stack to Docker Swarm to address these issues. Docker Swarm allows for individual container scaling and deployment configuration, meaning we can scale the main application to two containers. Docker Swarm places a load balancer in front of these containers and uses round-robin load balancing to distribute traffic between them.

The CI/CD pipeline then informs the swarm cluster that a new image is available. Docker Swarm automatically replaces one container with the updated image and waits until the new container is stable and healthy before repeating the process for the remaining container. This process occurs without modifying the other applications in the stack, such as the database, monitoring, and logging services. As a result, downtime for end users is effectively eliminated when updating the application.

7 Security

7.1 Security Assessment

7.1.1 Risk Identification

7.1.1.1 Assets

- **Web application**
- **User data** in PostgreSQL
- **Server**
- **Secrets**
- **CI/CD pipeline**
- **Monitoring stack**

7.1.1.2 Threat sources

- External attackers
- Malicious users
- Supply-chain compromise (Go modules, base images, GitHub Actions)
- Developer mistakes
- Cloud / hardware failure

7.1.1.3 Risk scenarios

- **R1**: Attacker brute-forces `POST /login` to take over user accounts.
- **R2**: Attacker uses the hardcoded simulator API token in `internal/web/router.go` to post or read messages as any user.
- **R3**: Attacker scrapes or floods the API (`/msgs?no=…`) and takes down the server.
- **R4**: Attacker performs CSRF against `/add_message` or `/{username}/follow` – no CSRF protection on form POSTs.

- **R5**: Grafana is publicly readable (`GF_AUTH_ANONYMOUS_ENABLED=true`) and Kibana is exposed on port 5601 – information disclosure.
- **R6**: Hetzner server is lost or compromised.
- **R7**: A developer accidentally commits `.env`.
- **R8**: A third-party GitHub Action used in CI is compromised and obtains `SSH_PRIVATE_KEY` / `SERVER_IP`.
- **R9**: Internal services expose host ports in `docker-compose.yml`.
- **R10**: Short and simple passwords can be brute forced in seconds.

7.1.2 Risk Analysis

7.1.2.1 Likelihood & Impact

7.1.2.1.1 Scales

- **Likelihood:** 1 (Rare), 2, 3, 4, 5 (Almost certain)
- **Impact:** 1 (Negligible), 2, 3, 4, 5 (Severe)

Risk scenario	Likelihood	Impact	Score
R1	4	4	**16**
R2	5	3	**15**
R6	3	5	**15**
R4	4	3	**12**
R5	4	3	**12**
R10	3	4	**12**
R7	2	5	**10**
R8	2	5	**10**
R3	3	3	**09**
R9	5	3	**09**

7.1.2.2 Risk Matrix

			Impact		
Likelihood	1	2	3	4	5
5	—	—	R2, R9	—	—
4	—	—	R4, R5	R1	—
3	—	—	R3	R10	R6
2	—	—	—	—	R7, R8
1	—	—	—	—	—

7.1.2.3 Discussion Both R9 and R10 have been addressed. We removed the `ports:` exposures for Grafana and Kibana from `docker-compose.yml`, so neither service exposes a host port anymore – they remain reachable only through Caddy on `grafana.kulturbase.dk` and `kibana.kulturbase.dk`.

We have also adressed R2 by moving the authorization token into `.env`.

For R10, we replaced every service credential in `.env` with random strings generated with `openssl`.

We prioritised R9 and R10 because they were the most exposed and relatively cheap to fix.

7.2 Firewall

We use both Hetzner's built-in firewall and `ufw` on the server. Only ports `80`, `443`, and `22` are open to incoming traffic.

7.3 TLS

We needed HTTPS in front of the app, and the lecture recommended using Nginx with Certbot. We chose Caddy instead because it combines both into a single tool: it automatically requests and renews Let's Encrypt certificates and redirects HTTP traffic to HTTPS. Our TLS setup is a single Caddyfile that routes `kulturbase.dk` and the `grafana.` and `kibana.` subdomains to their respective containers.

7.3.1 Container Hardening

7.3.2 Not running as root

We have added a `USER` directive to our Dockerfile to prevent the images running as `root`. We have additionally run `docker scout` on our images which revealed no critical vulnerabilities. Our base images are up-to-date with versions `golang:1.25` and `alpine:3.23`, neither of which has any known vulnerabilities.

7.3.3 Image scan for vulnerabilities

The initial scan of our `minitwit` image using `docker scout cves -only-severity critical,high` revealed 4 vulnerabilities: 2 critical in the `github.com/jackc/pgx/v5` Go dependency and 2 high in the Alpine 3.19 image. The following fixes were applied:

- Updated `pgx/v5` from `v5.6.0` to `v5.9.2`
- Updated `alpine:3.19` to `alpine:3.23`

A follow-up scan reports 0 critical and 0 high risk vulnerabilities.

7.4 Enhance your CI Pipelines with multiple security related static analysis tools

CI runs CodeQL on every push and PR to `main`. The deploy job depends on `codeql`, and branch protection requires the code scanning check to pass before merge, so any finding above a specific threshold blocks merge and deployment.

CI also runs Trivy on every push and PR to `main`: the `image_scan` job builds the container and scans it for fixable HIGH/CRITICAL vulnerabilities in OS packages and Go modules. The deploy job depends on `image_scan`, so any vulnerable image is blocked from deployment.

8 Reflections

Reflecting on the project at hand, a couple of learnings stand out. When we started refactoring the legacy version of the application, we transitioned to a Golang code-base. Picking a language unfamiliar to the majority of us made the refactoring process pretty steep. Refactoring the unit-tests of the legacy app, for example, demanded the use of Go-libraries that functioned vastly different than frameworks such as pytest (if one tries to run the code from the initial commit of test-files, 8bbbaff, it is clearly broken). This learning-by-doing approach, however, made the evolution of the program easier, as the time spent understanding the new code base, made any evolutionary steps more fluent.

The logging and monitoring parts of the operation process was one of the main differences to other software projects we have worked on. The simulation really forced us to actually prioritize backlog items for deployment in a more intentional matter, as a large amount of user's 'depended' on our service. In a software side-project, you can develop new features as you find them interesting, but with 100.000 users, an unoptimized http-handler quickly becomes a top priority. This also made maintenance more of a matter of urgency than previous projects we have worked on. Something breaking has consequences for a vast amount of users, and sometimes, that demanded late night debugging. All of this was only possible by our monitoring and logging frameworks, and the course has given us great insights into working with such in the future.

9 AI Acknowledgement

As per ITU guidelines on the use of Generative Artificial Intelligence, this appendix specifies how generative AI was used in the development of our project.

At the tail end of the project, it occurred to us that we did not properly credit the models used in our commits. Out of the ~ 180 commits to the repository, artificial intelligence was used in about 40%, as per the description in A.2

9.1 Which models were used?

This project has used Microsoft Copilot, ChatGPT-5, developed by OpenAI, and Claude.ai (Opus and Sonnet), developed by Anthropic.

9.2 How was AI used?

Apart from helping us format our `.tex` files, the large language models were mostly used for debugging and refactoring. The initial refactoring from Python to Go resulted in a lot of errors during the refactoring process, and the models were used to single out what problem had to be solved from the error codes.

Similarly, we used models to single out errors in our log files. When something broke in our deployment pipeline, it was not always easy to find the breaking point in a log file thousands of lines long. Using the models helped us track the breaking point of the deployment-deployment pipeline, and made pinpointing the exact cause for a quick fix much easier.

In cases where an LLM-generated code was used, we made sure to assess any code blocks before use, and kept our architecture and security in mind. When it comes to hindrances regarding AI usage, some LLM-generated code was clearly not up-to-standard and required extra review effort.

All in all, the use of LLM's helped us cut the time debugging any failures, and freed up time to work on strengthening our operations and evolution efforts.